

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_47057438-9c9\agent-a90f484e70c865137.jsonl`  
- SHA-256: `faf73b6f013449b87644705cdcb3ce0a5e491edd58aa11a977581af960a0dc2d`  
- Source modified: `2026-06-16T03:08:21+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_47057438-9c9`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T03:05:55.399Z)

DESIGN COMPONENT: The protobuf fixture SCHEMA + test-harness integration + determinism, building on the EXISTING infra in the grounding (eval_baselines/, regression.py, fusion_golden->*golden_features.json, fusion_compare/ablation, the skipif litmus tests). Sketch a .proto schema for a per-fixture record (anonymized id, fps, frame count, trajectory points, per-signal feature columns, GT intervals, expected/snapshot output segments, schema_version, label_version, detector tag). Describe: codegen & where generated code lives; how an auto-discovered parametrized pytest loads each committed fixture, REPLAYS it through the real segmenter/fusion code (the video-free seam) and asserts BOTH a characterization/snapshot match (catch any refactor side-effect) AND a quality-floor vs GT; how to keep serialization deterministic & cross-OS stable (sorted fields, fixed float formatting, newline normalization); schema/label versioning + migration. Reuse, don't reinvent ? say exactly what to extend.

LEGAL MEMO (JSON):

```
{  
  "executiveSummary": "INTERNAL MEMO ? general legal information, NOT legal advice; retained India + EU/UK counsel sign-off required before any public commit. QUESTION: can the project commit DERIVED numeric data (per-frame shuttle (x,y) coordinates, rally start/end time-intervals, audio/person/court feature columns, ground-truth intervals) into a public GitHub repo so external contributors can run the rally-segmentation regression suite WITHOUT the videos? BOTTOM LINE: Yes, but only under conditions, and the analysis FORKS on data provenance. The OUTPUT data itself is, across all three jurisdictions, essentially uncopyrightable FACT/measurement ? it does not inherit the source video's copyright (US: Feist + 17 USC 102(b) + NBA v. Motorola; India: EBC v. D.B. Modak, no sui generis right; EU: Football Dataco v. Yahoo ? copyright protects only original selection/arrangement, never the numbers). So a thin compilation copyright at most attaches to a creative schema, never to the coordinates. BUT copyright is the easy axis and NOT where the risk lives. The real exposure is fourfold and provenance- and stream-dependent: (1) the ACT of ingesting/extracting from the source video needs its own lawful basis ? owned/permissioned video is clean; a US fair-use defense for analytic extraction is now CONTESTED, not \"strongly favorable,\" after Thomson Reuters v. Ross (2025) and the US Copyright Office May-2025 report, and unlawful acquisition independently destroys the defense (Bartz v. Anthropic, ~$1.5B settlement); (2) CONTRACT / Terms-of-Service can bar extraction and redistribution even where copyright cannot reach the facts (ProCD; hiQ v. LinkedIn; YouTube ToS bars scraping AND \"harvest information that might identify a person (for example... faces)\") AND redistribution) ? and the remedy is injunction + forced deletion of the committed dataset, not nominal damages; (3) EU/UK sui generis DATABASE RIGHT may subsist in self-derived match-event/coordinate streams because, on the leading Football Dataco v. Sportradar reasoning, recording pre-existing on-court facts you do not control is \"OBTAINING,\" not \"creating\" ? so the project's comfortable \"we created it\" premise is genuinely contestable; (4) PRIVACY ? person/pose/gait streams and anything involving MINORS are the highest-risk and several should NOT be published at all. The cleanest defensible path: commit ONLY owned-or-permissioned-source, minors-excluded, person/pose/gait-EXCLUDED (shuttle + abstract rally-interval + non-biometric feature) data, stripped of every source-attribution field (no filename, no video_id MD5, no match metadata), under a no-extraction-from-third-party-source and no-redistribution hygiene posture, with optional encryption only as a barrier against
```

the no-key public (it cannot bind a contributor who runs the tests).",

"derivedDataCoreVerdict": "The derived numeric data DOES NOT inherit the source video's copyright in any of the three jurisdictions ? this is the one firmly-settled conclusion. Per-frame shuttle (x,y) coordinates, rally start/end timestamps, and audio/person/court feature values are uncopyrightable FACTS/measurements about a recorded physical event: discovered, not authored. US: Feist (facts never copyrightable; \"originality, not sweat of the brow, is the touchstone\"), 17 USC 102(b) (no copyright in any \"discovery\"), NBA v. Motorola (facts generated by a sporting event are uncopyrightable \"purely factual information which any patron... could acquire from the arena\" even though the broadcast is protected). A coordinate/timestamp table is also NOT a 17 USC 101 \"derivative work\": a derivative work recasts the original's protected EXPRESSION (framing, editing, imagery, commentary), and a number table carries none of it. India: EBC v. D.B. Modak ? copyright needs \"skill and judgment,\" protects only original arrangement, not facts; no EU-style sui generis right exists. EU: Football Dataco v. Yahoo (C-604/10) ? database copyright protects only an original selection/arrangement; \"the intellectual effort and skill of creating that data are not relevant\" and sweat-of-the-brow cannot ground it. CEILING ON PROTECTION: the MOST that can attach is a THIN compilation copyright in a genuinely creative SELECTION/ARRANGEMENT (e.g., a curated schema) ? never in the raw numbers, and a mechanical/exhaustive layout (every frame, every rally) likely has too little originality even for that (Feist's \"garden-variety listing\" problem). CRITICAL SEPARATION: the STATUS of the output data (settled: unprotected facts) is a different question from the ACT of extraction (transiently copying the video to compute the data), which can independently infringe (Sega v. Accolade: intermediate copying can infringe even when the end product reproduces no protected expression) and which needs its own license or fair-use basis. NON-COPYRIGHT FENCES survive precisely BECAUSE the data is uncopyrightable: enforceable no-extraction ToS (ProCD), narrow hot-news misappropriation (NBA v. Motorola), and the EU/UK sui generis database right. So \"the numbers are free facts\" answers the copyright question and ONLY the copyright question.",

"byJurisdiction": [
 {
 "jurisdiction": "India (PRIMARY ? owner is Bangalore-based, commercialization intended)",
 "copyright": "Derived facts are NOT copyrightable as such: EBC v. D.B. Modak (2008 SC) requires 'skill and judgment' and protects only original arrangement, not facts; India has NO EU-style sui generis database right. BUT this is not as clean as it looks: Indian High Courts protect DATABASES/COMPILATIONS as 'original literary works' under s.2(o) on a low 'skill, labour and judgment' threshold ? Burlington Home Shopping v. Rajnish Chibber, V. Govindan v. E.M. Gopalakrishna Kone, and the OLX v. Padawan order itself (which held a scraped dataset to be a protectable 'proprietary database'). So if numbers are LIFTED from a third party's structured dataset/feed, copying its selection/arrangement can infringe even though the raw facts are free. Self-derived-from-pixels data avoids this; copying an official stats compilation does not.",
 "databaseRight": "No sui generis database right exists in India ? a genuine permissiveness advantage over the EU/UK. Protection runs only through copyright (compilation, above) or contract/IT-Act.",

"contractToS": "The LIVE risk, not copyright. Breach of anti-scraping ToS plus IT Act s.43 (Government has affirmatively argued public-data scraping 'may be in breach of section 43') and s.66 layering CRIMINAL exposure (up to 3 years / Rs 5 lakh). Enforceability of anti-scraping ToS is untested in Indian courts. OLX v. Padawan was decided AGAINST the scraper; its 'private use' line is obiter, not a safe harbor. Trade-secret/confidentiality for the owner's OWN corpus runs NOT through any US-style DTSA (India has no trade-secret statute) but through the Indian Contract Act 1872, equity, common-law breach of confidence, and IT Act s.72 ? and publication still forfeits the moat under that doctrine.",

"privacy": "DPDP Act 2023 governs personal data of identifiable natural persons; genuinely anonymized numbers fall outside it, but the s.3(c)(ii) 'publicly available data' carve-out is CONTESTED (Meity/Minister has said scrapers must still meet DPDP consent obligations). MINORS are stricter than the EU: 'child' = under 18 (s.2(f)); s.9(1) requires VERIFIABLE parental consent (DPDP Rules 2025 Rule 10 ? real ID/age verification, not a checkbox); s.9(2) bars detrimental processing; s.9(3) PROHIBITS tracking/behavioural monitoring/targeted ads of children ? subject to narrow Rule 12/Fourth-Schedule class/purpose exemptions that do NOT cover an amateur highlight indexer, so the ban DOES bite this project. A persistent per-player pose/gait signature is, on the conservative reading, prohibited behavioural monitoring; 'tracking' is undefined in the Act, so this is the defensible-conservative, not the certain, reading.",

"netVerdict": "MORE PERMISSIVE than the EU on the database-right axis, but NOT 'lawful on its face.' Defensible to publish ONLY genuinely-anonymized, independently-derived (not lifted from a third-party compilation) numbers, with NO minors tracked and accepting that anti-scraping-ToS + IT Act s.43/s.66 exposure for any third-party-sourced ingestion is unresolved.

Owned-source + minors-excluded + no-person/pose/gait = clean."

```
  },
  {
    "jurisdiction": "United States (repo + product are global; US fair-use & contract law are
the most-developed)",
    "copyright": "Output data does not inherit the video's copyright (Feist; 102(b); NBA v.
Motorola); not a 101 derivative work; thin compilation copyright at most. The EXTRACTION act is
the open question: fair use for analytic, non-expressive copying was historically favorable
(Authors Guild v. Google; Google v. Oracle) but is now CONTESTED ? Thomson Reuters v. Ross (D.
Del. 2025) REJECTED the 'copy an expressive work just to reach unprotected facts/data' theory,
distinguished the functional-CODE cases (Sega/Oracle), found the use non-transformative, and
found Factor-4 harm in a derivative 'data to train AIs' market the plaintiff had not even
entered; the US Copyright Office (May 2025) rejected 'non-expressive therefore inherently
transformative' and backs a market-dilution harm theory. If the dataset competes with a rights-
holder's own data/analytics market, fair use may FAIL. Unlawful source acquisition independently
defeats the defense regardless of how transformative the end use is (Bartz v. Anthropic, 2025;
~$1.5B settlement).",
    "databaseRight": "No sui generis database right in the US (Feist rejected sweat-of-the-
brow). Competitors stay free to copy raw facts ? subject to the non-copyright fences below.",
    "contractToS": "The dominant US risk. Post-Van Buren the CFAA no longer reaches mere
purpose-violations of accessible/public data (though footnote 8 RESERVED whether contract/policy
limits can still ground CFAA liability ? the federal-crime route is narrowed, not dead), so
platforms enforce via CONTRACT. ToS is an enforceable contract that bites where copyright/CFAA
fail (ProCD; hiQ v. LinkedIn ? breach-of-contract summary judgment + $500k consent judgment +
injunction + ORDER TO DESTROY scraped-derived data). Binding turns on assent: a logged-
in/account/API-key or genuine clickwrap user is bound (hiQ; any YouTube-API user); a purely
logged-out scraper under browswrap-only may not be (Meta v. Bright Data) ? and thin 'sign-in
wrap' flows can fail for want of conspicuous notice. YouTube ToS verbatim bars automated access,
'harvest any information that might identify a person (for example... faces)', and
redistribution/commercial reuse. REDISTRIBUTION (committing the dataset publicly) is the single
highest-risk act and the remedy is injunction + forced deletion, not nominal damages.",
    "privacy": "GDPR-style analysis below applies to global users; US state biometric statutes
(e.g., Illinois BIPA ? not separately briefed here) and publicity/personality rights are
additional fences for face crops and identifiable-player gait. Faces in any committed imagery =
identifying personal data; do not publish.",
    "netVerdict": "Output data publishable as facts ONLY if (i) source video was
owned/permissioned (or extraction has a defensible fair-use basis you do NOT over-rely on), (ii)
no enforceable ToS bars extraction/redistribution of the ingested source, and (iii) no
person/pose/gait/face/minor stream is included. Treat third-party-ToS-sourced derived data as
NOT publicly committable."
  },
  {
    "jurisdiction": "EU / UK (product/repo global; English-language highlight market makes UK
a likely target)",
    "copyright": "Database copyright protects only an original selection/arrangement, never
the numbers, and never sweat-of-the-brow (Football Dataco v. Yahoo, C-604/10). Self-derived
coordinates are facts, not a copied original structure ? so copyright is a low axis.",
    "databaseRight": "The genuinely contestable axis. The sui generis right protects
investment in OBTAINING/verifying/presenting pre-existing data, NOT in CREATING it (BHB v.
William Hill C-203/02; Fixtures Marketing) ? which the project reads as 'we create coordinates
frame-by-frame, so we are clear.' THAT READING IS OVERCONFIDENT: in Football Dataco v.
Sportradar the live match-event data (goals/scorers) was held to be OBTAINED, because the
recorder captured pre-existing facts ('events taking place on the pitch') it did NOT control. A
badminton rally/shuttle position is the same kind of pre-existing on-court fact recorded rather
than authored, so a rights-holder can argue the right SUBSISTS in our streams. Infringement is
now governed by CV-Online Latvia (C-762/19, 2021): even a substantial-part taking is actionable
only where it adversely affects the maker's ability to recoup investment, balanced against
information flow/competition ? a fact-intensive harm test, not a clean 'no protected part =
safe' rule. Sportradar (C-173/11) also defeats the server-location dodge for EU-targeted re-
utilisation. UK retained its own sui generis right and pre-2021 CJEU case law, so UK-targeted
products carry AT LEAST equal, arguably greater, risk; a non-UK/EEA maker may not even hold the
right defensively, which does nothing to reduce exposure as a potential infringer.",
    "contractToS": "Ryanair v. PR Aviation (C-30/14): where a database enjoys NEITHER
copyright NOR the sui generis right, the Directive's user-protections (incl. no-contracting-out)
do not apply, so the source's T&Cs/licence can lawfully BAN automated extraction and
```

redistribution and bind anyone who agreed ? the dominant residual risk. Trap: the licence that legitimises INGESTION of the footage is often the very instrument that contractually bans extraction/redistribution of derived data.",

"privacy": "GDPR: shuttle/object coordinates are NOT personal data in isolation (Art.4(1)) ? but coordinates published as part of an identifiable named match are part of a personal-data operation. Person bounding boxes that single out a human are ordinary personal data, NOT Art.9 biometric UNLESS used to uniquely identify (EDPB Guidelines 3/2019). Pose/gait is the danger zone: gait re-identifies even with faces blurred and skeleton 'anonymization' leaves gait identity intact (Weiss et al., Sensors 2025) ? Art.9 special-category the moment used to uniquely identify; explicit-consent basis required. 'Anonymous' is RELATIVE post EDPS v. SRB (CJEU C-413/23 P, 4 Sept 2025): a stream can be non-attributable to a recipient lacking the source yet remains PERSONAL DATA to the holder who can re-run the detector. Minors: Art.8 (consent floor 13716); any biometric/special-category processing of a child stacks Art.9.",

"netVerdict": "Do NOT assume self-derivation defeats the database right ? treat self-derived event/coordinate streams as potentially 'obtained' data and avoid taking in a way that could reconstitute a rights-holder's dataset or harm their feed's monetisation. Read every source's T&Cs for extraction/redistribution clauses. Keep the footage-copyright analysis (highlight reels) strictly separate. Person/pose/gait/face = do not publish without an Art.9 explicit-consent basis. Get EU + UK jurisdiction-specific advice before redistributing numeric streams at scale."

}

],

"provenanceFork": "PROVENANCE IS THE DECISIVE FORK ? the legal posture of an identical numeric file flips entirely on where the source video came from. FORK A ? OWNED / OWNER-RECORDED (or fully-permissioned/licensed-for-this-use) SOURCE: This is the clean path and the only one safe to commit publicly. No third-party copyright in the source bars the extraction; no third-party ToS governs the owner's own recording (with two caveats below); the EU/UK database-right 'obtained vs created' debate still applies in theory but there is no adverse rights-holder to assert it against the owner's own footage. The analysis collapses to: (1) a TRADE-SECRET / competitive-moat CHOICE (US framing: 18 USC 1839 DTSA requires secrecy measures + value-from-secrecy; the OPERATIVE framing for this Bangalore owner is Indian breach-of-confidence / Contract Act 1872 / IT Act s.72, NOT the DTSA) ? publishing the data forfeits whatever moat it carried, which is the owner's strategic call to make, weighed against the stated community/benchmark/paper-co-authorship aims; and (2) PRIVACY ? the owner's own footage still contains identifiable people, so DPDP/GDPR and the minors rules below still bind, and person/pose/gait/face streams are still off-limits. TWO CAVEATS that narrow Fork A: (a) 'owner-recorded' is clean ONLY for the owner's OWN non-ticketed/private sessions; footage the owner SHOT AT a third party's event (tournament, club match) carries live exposure ? a ticket is a contract of admission that routinely bars recording/redistribution, and venue/organiser/broadcaster rights (e.g., BWF/league terms, Rome Convention rebroadcast rights) attach independently of any platform ToS; (b) any third-party-collected footage in the corpus (the project's golden set may mix sources) taints those records and pushes them into Fork B. FORK B ? THIRD-PARTY-SOURCED (broadcast / YouTube / scraped / downloaded) SOURCE: Derived data from this provenance SHOULD NOT be committed to a public repo. Even though the numbers are uncopyrightable facts, (1) the EXTRACTION act needs a license or a now-CONTESTED US fair-use defense, and unlawful acquisition (e.g., scraping against ToS, downloading from a platform that bars it) independently sinks the defense (Bartz v. Anthropic); (2) the source's ToS (YouTube et al.) typically bars automated extraction AND redistribution, and committing the derived dataset publicly IS redistribution ? the highest-risk act, remediable by injunction + forced deletion of the committed data (hiQ); (3) re-utilisation/extraction of a substantial part may infringe the EU/UK sui generis database right if the source ties back to a protected feed. PRACTICAL RULE FOR THE REPO: gate the committed corpus to FORK A only. Tag every golden-set record with a verified provenance flag at ingestion; only owned/permissioned-source, non-event-ticketed, minors-excluded, person/pose/gait-excluded records are eligible for the public commit. This maps directly onto the existing split: eval_baselines/ (aggregate JSON snapshots) is already committed precedent and is provenance-agnostic if it carries no per-source attribution; *_golden_features.json are currently kept private (docs/GOLDEN_DATA_SHARING.md) precisely because they derive from a 'Proprietary corpus' ? the right move is to keep Fork-B-tainted features private and publish only a Fork-A, de-attributed subset.",

"privacyVerdict": "PUBLISH/NO-PUBLISH BY STREAM (the project's guardrails already point the right way; this hardens them). PUBLISH-OK (lowest risk): (1) SHUTTLE (x,y) coordinates + Visibility ? describe an inanimate object, NOT personal data on their own (GDPR Art.4(1); WP136 content/purpose/result test) ? PROVIDED they are NOT joined to anything that singles out a named player and NOT published as an identifiable named match (the surrounding personal-data obligations survive even when the coordinate column alone is object-data). (2) Abstract RALLY

start/end time-INTERVALS and ground-truth intervals stripped of match/player identity ? facts about event timing. (3) Non-biometric AUDIO-ONSET, COURT-BOUNDS, and aggregate PERSON COUNT/feature columns used ONLY for rally detection and DETACHED from any identity ? ordinary low-risk data with normal hygiene; NOT Art.9 because the purpose is detect/categorize, not uniquely identify (EDPB Guidelines 3/2019). CONDITIONAL ? only with an Art.9 explicit-consent basis (and effectively never for this v1): PERSON bounding-box coordinates that persistently single out and follow a specific player across rallies ? the purpose flips to unique identification and Art.9 attaches; this architecture is PRONE to that flip, so treat per-player tracking as a tripwire. DO NOT PUBLISH (high risk, exclude from any committed stream): (4) POSE keypoints / SKELETON / GAIT signatures ? gait is a behavioural biometric that re-identifies people even with faces blurred, and pose 'anonymization' provably leaves gait identity intact (Weiss et al., Sensors 2025, 80?95% gait-recognition on anonymized data); raw pose/gait tracks are NOT reliably anonymous, are personal data when usable to single out a player, and become Art.9 special-category once used to uniquely identify ? this matches the project's existing 'gait/biometrics strictly FUTURE (GDPR Art.9)' guardrail. (5) Any FACE CROP or identifiable IMAGERY ? identifying personal data under GDPR and DPDP, and barred by YouTube ToS's face-harvesting clause; do not commit imagery at all. (6) Anything that joins a NAMED PLAYER to a gait/pose signature. MINORS FLIP EVERYTHING: under DPDP a 'child' is anyone under 18; processing ANY child's data needs VERIFIABLE parental consent (Rule 10 ? real ID/age check, not a checkbox), must not cause detrimental effect, and s.9(3) PROHIBITS behavioural monitoring/tracking of children (Rule 12/Fourth-Schedule exemptions do not cover this project, so the ban applies). A persistent per-player pose/gait signature is, conservatively, prohibited monitoring. CLEANEST PATH for any footage containing minors: EXCLUDE minors entirely from any persisted derived stream beyond inanimate shuttle/interval data ? consistent with the existing 'EXCLUDE MINORS from the training pool' guardrail ? or obtain Rule-10 verified parental consent and keep only de-identified object/coordinate data. NON-ATTRIBUTABILITY ARCHITECTURE (the 'secret file' question): Non-attributability is a property of the (data, holder, environment) triple, not of the timeseries. A per-video trajectory/feature stream is effectively a FINGERPRINT (de Montjoye: 4 points re-identify 95% of 1.5M people; k-anonymity collapses on rich quasi-identifiers), so a deterministic detector re-run on a candidate video can correlate-match it to its source ? BUT only if a correlatable reference EXISTS. PUBLIC source: a permanent cheap reference exists, re-linkage is 'reasonably likely', so a raw public-sourced stream should be treated as ATTRIBUTABLE; only heavy aggregation, perturbation/differential privacy, synthetic data, or a non-public enclave (NIST SP 800-188) breaks correlatability. PRIVATE/owner-recorded-and-unpublished source: there is no external reference to correlate against, so source-linkage by an EXTERNAL adversary is not 'reasonably likely' ? BUT it is WRONG to call this 'anonymous by default' (EDPS v. SRB, CJEU C-413/23 P, 4 Sept 2025): to the holder who can re-run the detector, it REMAINS personal data; it is merely RELATIVELY non-attributable to outsiders, and that protection is conditional, revocable (publish or leak the source and the link snaps back), and forward-looking (regulators say assume the data environment grows). HARD ENGINEERING IMPLICATION: the committed fixtures must NOT carry the source filename, match metadata, or the video_id (which is the MD5 of the source file ? using it as a public key would FINGERPRINT/confirm the source the instant anyone holds the file). Use an opaque, random, non-reversible surrogate id per record. Strip the 'name' field in *_golden_features.json. The lead's established constraint is correct: a contributor who RUNS the tests can necessarily read the plaintext fixtures and any key on their machine (DRM/analog-hole limit), so secrecy-from-the-contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-isolation, with encryption useful ONLY as a barrier against the no-key public, never against a running contributor.",

"conditionsForSafePublicCommit": [

"PROVENANCE GATE: commit ONLY derived data from owner-recorded / fully-permissioned source video (Fork A). Tag every golden-set record with a verified provenance flag at ingestion; exclude any third-party-sourced (broadcast/YouTube/scraped/downloaded) record from the public commit. For owner-SHOT footage of a third party's event, first clear the admission ticket / venue / organiser / broadcaster terms ? only owner's own non-ticketed sessions are unconditionally clean.",

"SOURCE-LICENCE/ToS CHECK: for every source that touched the committed corpus, confirm in writing that its licence/ToS does NOT bar automated extraction OR redistribution of derived data. Do not rely on US fair use for the extraction step ? it is now contested (Thomson Reuters v. Ross; USCO May-2025) and unlawful acquisition independently defeats it (Bartz v. Anthropic). Redistribution (the public commit) is the single highest-risk act.",

"MINORS EXCLUSION: exclude all footage containing minors (under 18, per DPDP s.2(f)) from any persisted/committed derived stream beyond inanimate shuttle/interval data ? unless Rule-10 verifiable parental consent is held AND only de-identified object/coordinate data is kept. s.9(3)'s behavioural-monitoring ban means a persistent per-player pose/gait signature on a minor

is prohibited.",

"BIOMETRIC EXCLUSION: do NOT publish pose keypoints, skeleton tracks, gait signatures, face crops, identifiable imagery, or any named-player-to-movement-signature mapping. Restrict the committed feature columns to shuttle (x,y)+Visibility, abstract rally/GT time-intervals, and non-biometric audio/court/aggregate-person features used only for rally detection and detached from identity.",

"DE-ATTRIBUTE: strip every source-identifying field from the committed fixtures ? source filename, match/event/player metadata, capture dates, and especially the video_id (MD5 of the source file). Replace with an opaque, random, non-reversible surrogate record id. The 'name' field in *_golden_features.json must be removed/aliased.",

"NON-ATTRIBUTABILITY POSTURE: because per-video trajectories are fingerprints correlatable to a source video, achieve non-attributability by keeping the SOURCE private (Fork A, unpublished) ? which makes external source-linkage not 'reasonably likely' ? and/or by coarsening/aggregating/perturbing where feasible. Document the (data, holder, environment) assessment and re-test it as the public-video environment grows (ICO/OVIC forward-looking duty). Treat the streams as personal data IN THE OWNER'S OWN HANDS regardless (EDPS v. SRB).",

"USELESS-IN-ISOLATION + ENCRYPTION-AS-PUBLIC-BARRIER ONLY: keep the committed fixtures useless without the regression harness + schema (abstract intervals + opaque ids, no imagery, no source pointer). Use encryption only as a barrier against the no-key public; do not represent it as protecting against a contributor who runs the tests (they can read plaintext + key locally ? the lead's established DRM/analog-hole constraint).",

"PRECEDENT-CONSISTENT PLACEMENT: publish via the already-committed eval_baselines/ pattern (aggregate, provenance-agnostic JSON snapshots) for headline regression baselines; keep Fork-B-tainted or biometric-bearing *_golden_features.json private (per docs/GOLDEN_DATA_SHARING.md) and publish only a Fork-A, de-attributed, minors-excluded, biometric-free subset of features needed to run the suite.",

"DPDP HYGIENE: ensure any person-derived numeric column that could single out a player is genuinely identity-detached; do not rely on the DPDP s.3(c)(ii) 'publicly available data' carve-out for non-anonymized personal data (it is contested). Maintain purpose-limitation: features persisted for rally detection must not be repurposed for player identification.",

"COUNSEL SIGN-OFF: obtain retained India counsel (DPDP s.9/Rules, IT Act s.43/s.66, breach-of-confidence) AND EU/UK counsel (sui generis database right 'obtained vs created', CV-Online harm test, Ryanair contract) review of the final committed subset and the source-provenance log BEFORE the first public commit. This memo is general information, not advice."

],

"redFlags": [

"OVERCONFIDENCE TRAP #1 ? 'we created the data, so the EU/UK database right cannot apply.' Football Dataco v. Sportradar held live match-event data was OBTAINED (pre-existing on-court facts the recorder did not control), exactly analogous to a recorded badminton rally/shuttle position. A rights-holder can credibly argue the sui generis right subsists in our self-derived streams. Do not bank on the 'created' characterisation.",

"OVERCONFIDENCE TRAP #2 ? 'analytic extraction is strongly favorable fair use (US).' Thomson Reuters v. Ross (2025) REJECTED the 'copy an expressive work to reach unprotected facts' theory and found Factor-4 harm in a derivative data market; the USCO (May 2025) rejected 'non-expressive = inherently transformative.' If the dataset competes with a rights-holder's data/analytics market, fair use may fail. The extraction step is litigable, not settled.",

"OVERCONFIDENCE TRAP #3 ? 'owner's own recordings have no contract problem.' False for footage shot at a third party's event: the admission ticket is a contract barring recording/redistribution, and venue/organiser/broadcaster rights (BWF/league/Rome Convention) attach independently. Only the owner's own non-ticketed sessions are unconditionally clean.",

"OVERCONFIDENCE TRAP #4 ? 'private/unpublished source = anonymous by default.' Per EDPS v. SRB (CJEU 2025) the stream REMAINS personal data to the holder who can re-run the detector; it is only RELATIVELY non-attributable to an external adversary lacking the source, and that is conditional, revocable, and forward-looking. Misclassifying it as 'anonymous' is the exact GDPR error the controller's-perspective holding exists to prevent.",

"FINGERPRINTING / video_id leak: the project's video_id is the MD5 of the source file. Committing it (or any source filename/match metadata) as a public key would let anyone holding the candidate video CONFIRM the source instantly ? defeating non-attributability. Must be stripped and replaced with an opaque surrogate.",

"REDISTRIBUTION REMEDY is not nominal damages: the operative relief in scraping/ToS disputes is INJUNCTION + FORCED DELETION of the derived dataset (hiQ ordered destruction of all scraped-derived code/data). A public commit that breaches a source ToS can be ordered taken down and deleted ? a real, not theoretical, downside.",

"POSE/GAIT mislabeled low-risk: skeleton 'anonymization' leaves gait identity intact (80?95% re-ID), so any pose/gait stream is NOT reliably anonymous and is Art.9-adjacent. Excluding it is

not optional polish ? it is a guardrail.",

"MINORS + behavioural monitoring: DPDP s.9(3) bars behavioural monitoring of under-18s and the Rule 12/Fourth-Schedule exemptions do not cover this project. A persistent per-player pose/gait signature on a minor is, conservatively, a prohibited act with criminal-adjacent regulatory exposure ? exclude minors from any persisted stream beyond inanimate shuttle/interval data.",

"MIXED-PROVENANCE corpus: the golden set may blend owned and third-party footage. A single Fork-B-tainted record contaminates the public commit. Provenance must be verified per-record, not assumed corpus-wide.",

"WRONG-LAW trade-secret framing: analysing the owner's moat under US 18 USC 1839 (DTSA) is a jurisdictional error ? the Bangalore owner's protection runs through Indian breach-of-confidence / Contract Act 1872 / IT Act s.72. The publish-forfeits-the-moat intuition survives, but the operative doctrine is different."

],

"counselQuestions": [

"EU/UK sui generis database right: on the Football Dataco v. Sportradar 'obtained vs created' reasoning, does the right subsist in our self-derived shuttle-coordinate / rally-event streams, and if so does redistributing a de-attributed regression subset 'extract/re-utilise a substantial part' and cause CV-Online-Latvia-type harm to any rights-holder's feed? (EU AND UK, given likely English-language market.)",

"US fair use for the EXTRACTION step, post-Thomson Reuters v. Ross and the USCO May-2025 report: is transient analytic copying of a source video to compute rally/coordinate features defensible where the resulting dataset could be seen to compete in a 'data/analytics' market ? and does it matter that we only ever publish numbers, never the footage?",

"India: are anti-scraping/ToS terms enforceable in Indian courts, and what is the realistic IT Act s.43 (civil) / s.66 (criminal) exposure for ingesting third-party footage to derive features ? and does the pending ANI v. OpenAI judgment (reserved, ~April 2026) change the fair-dealing/TDM analysis?",

"India DPDP s.9 + Rules 10/12: for a corpus that may contain players under 18, what verifiable-parental-consent and exemption analysis applies, and does a persistent per-player pose/gait (or even per-player rally-tracking) feature constitute prohibited 'behavioural monitoring/tracking' of a child under s.9(3)?",

"Owner-shot event footage: do the admission tickets / venue / organiser (e.g., BWF, local league) / broadcaster terms for the specific tournaments and clubs in our corpus bar recording, derivation, or redistribution of derived data ? and which records must therefore be excluded from the public commit?",

"Personal-data classification under GDPR + DPDP for the SPECIFIC committed feature columns: at what point do identity-detached person/court/coordinate features 'single out' a player (especially in a small known squad/roster), and does our de-attribution + opaque-surrogate-id scheme suffice to keep the published subset outside personal-data obligations for downstream recipients (per EDPS v. SRB)?",

"Trade-secret / breach-of-confidence (Indian law): given commercialization and possible paper co-authorship, what is the moat trade-off of publishing the Fork-A subset, and are there confidentiality/Contract-Act 1872 / IT Act s.72 steps to take before forfeiting secrecy on any owner-recorded derived data?",

"Source-licence audit: for each source category in the corpus (owner-recorded, permissioned partner, downloaded/scraped), confirm the licence/ToS expressly permits BOTH automated extraction AND public redistribution of derived numeric data ? and where silent or prohibitive, which records are ineligible.",

"Encryption + 'secret file' design: does adding encryption that a running contributor can necessarily decrypt create any misrepresentation/false-security or DPDP 'reasonable security safeguards' compliance question, and is non-attributability-via-private-source a defensible posture to assert to regulators given its conditional/revocable nature?",

"Cross-border data transfer: does committing any record that is (or may be argued to be) personal data to a global public GitHub repo implicate DPDP cross-border-transfer rules or GDPR international-transfer obligations, independent of the copyright/contract analysis?"

],

"citations": [

{

"name": "Feist Publications, Inc. v. Rural Telephone Service Co., 499 U.S. 340 (1991) ?
Justia",

"url": "https://supreme.justia.com/cases/federal/us/499/340/"

},

{

"name": "National Basketball Ass'n v. Motorola, Inc., 105 F.3d 841 (2d Cir. 1997) ?

```

Wikipedia case summary",
  "url": "https://en.wikipedia.org/wiki/National_Basketball_Ass%27n_v._Motorola,_Inc."
},
{
  "name": "17 U.S.C. § 102 ? BitLaw / U.S. Code",
  "url": "https://www.bitlaw.com/source/17usc/102.html"
},
{
  "name": "17 U.S.C. § 101 Definitions ? Legal Information Institute (Cornell)",
  "url": "https://www.law.cornell.edu/uscode/text/17/101"
},
{
  "name": "Sega Enterprises Ltd. v. Accolade, Inc., 977 F.2d 1510 (9th Cir. 1992) ? BitLaw",
  "url": "https://www.bitlaw.com/source/cases/copyright/Sega-Accolade.html"
},
{
  "name": "Authors Guild v. Google, Inc., 804 F.3d 202 (2d Cir. 2015) ? Justia",
  "url": "https://law.justia.com/cases/federal/appellate-
courts/ca2/13-4829/13-4829-2015-10-16.html"
},
{
  "name": "Google LLC v. Oracle Am., Inc., 141 S. Ct. 1183 (2021) ? U.S. Copyright Office
Fair Use Index summary",
  "url": "https://www.copyright.gov/fair-use/summaries/google-llc-oracle-am-inc-2021.pdf"
},
{
  "name": "Thomson Reuters Enterprise Centre GmbH v. Ross Intelligence Inc. (D. Del. 2025) ?
full opinion (BitLaw)",
  "url": "https://www.bitlaw.com/source/cases/copyright/Thomson-Reuters-Ross.html"
},
{
  "name": "Perkins Coie ? Fair Use Defense Failed in Thomson Reuters v. Ross",
  "url": "https://perkinscoie.com/insights/update/fair-use-defense-failed-thomson-reuters-v-
ross-jury-still-out-generative-ai"
},
{
  "name": "ArentFox Schiff ? Landmark Ruling on AI Copyright: Bartz v. Anthropic",
  "url": "https://www.afslaw.com/perspectives/alerts/landmark-ruling-ai-copyright-fair-use-
vs-infringement-bartz-v-anthropic"
},
{
  "name": "Authors Guild ? Bartz v. Anthropic Settlement (~$1.5B; piracy-based liability)",
  "url": "https://authorsguild.org/advocacy/artificial-intelligence/what-authors-need-to-
know-about-the-anthropic-settlement/"
},
{
  "name": "Mayer Brown ? U.S. Copyright Office (May 2025, Part 3) on AI training and fair
use",
  "url": "https://www.mayerbrown.com/en/insights/publications/2025/05/united-states-
copyright-office-weighs-in-on-fair-use-defense-for-generative-ai-training"
},
{
  "name": "ProCD, Inc. v. Zeidenberg, 86 F.3d 1447 (7th Cir. 1996) ? Wikipedia",
  "url": "https://en.wikipedia.org/wiki/ProCD,_Inc._v._Zeidenberg"
},
{
  "name": "EUR-Lex ? Case C-203/02 British Horseracing Board v William Hill (9 Nov 2004)",
  "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A62002CJ0203"
},
{
  "name": "Wolters Kluwer / Kluwer Copyright Blog ? Database Directive sports-data cases",
  "url": "https://legalblogs.wolterskluwer.com/copyright-blog/databases-sui-generis-
protection-and-copyright-protection/"
},
{

```



```

    "name": "EUR-Lex ? Case C-604/10 Football Dataco v Yahoo (1 Mar 2012)",
    "url": "https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=CELEX:62010CJ0604"
  },
  {
    "name": "ipcuria ? Case C-173/11 Football Dataco v Sportradar (18 Oct 2012)",
    "url": "https://ipcuria.eu/case?reference=C-173/11"
  },
  {
    "name": "ipcuria ? Case C-762/19 CV-Online Latvia v Melons (3 Jun 2021)",
    "url": "https://ipcuria.eu/case?reference=C-762%2F19"
  },
  {
    "name": "Bird & Bird ? CV-Online Latvia: CJEU complicates enforcement of database rights",
    "url": "https://www.twobirds.com/en/insights/2021/uk/cv-online-latvia-cjeu-complicates-
the-enforcement-of-database-rights"
  },
  {
    "name": "Simkins ? Database rights back on the sport radar (Floyd J: live match events
'obtained')",
    "url": "https://www.simkins.com/news/database-rights-back-on-the-sport-radar"
  },
  {
    "name": "LexisNexis UK ? Legal protection of databases in the UK (post-Brexit assimilated
sui generis right)",
    "url": "https://www.lexisnexis.com/en-gb/legal/guidance/legal-protection-of-databases-in-
the-uk"
  },
  {
    "name": "EUR-Lex ? Case C-30/14 Ryanair v PR Aviation (15 Jan 2015)",
    "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62014CJ0030"
  },
  {
    "name": "EBC v D.B. Modak (WIPO Lex / Supreme Court of India)",
    "url": "https://www.wipo.int/wipolex/en/judgments/details/2840"
  },
  {
    "name": "Ikigai Law ? Legality of data scraping in India (OLX v Padawan; IT Act s.43; ToS;
database copyright)",
    "url": "https://www.ikigailaw.com/article/263/legality-of-data-scraping-in-india"
  },
  {
    "name": "IAPP ? Scraping public data in India: innovation enabler or privacy threat? (Govt
s.43 position; DPDP s.3(c)(ii))",
    "url": "https://iapp.org/news/a/scraping-public-data-in-india-innovation-enabler-or-
privacy-threat-"
  },
  {
    "name": "CaseMine ? Burlington Home Shopping v Rajnish Chibber (database as literary
work)",
    "url": "https://www.casemine.com/commentary/in/recognition-of-database-compilation-as-a-
literary-work:-analysis-of-burlington-home-shopping-pvt-ltd.-v.-rajnish-chibber-&-anr./view"
  },
  {
    "name": "CaseMine ? V. Govindan v E.M. Gopalakrishna Kone (compilation copyright)",
    "url": "https://www.casemine.com/commentary/in/establishing-copyright-infringement-in-
dictionary-compilations:-v.-govindan-v.-e.m.-gopalakrishna-kone/view"
  },
  {
    "name": "S.S. Rana & Co. ? Data scraping issues in India (IT Act s.43/s.66 criminal
exposure)",
    "url": "https://ssrana.in/articles/data-scraping-issues-india/"
  },
  {
    "name": "The New Publishing Standard ? Delhi HC reserves judgment in ANI v OpenAI (April
2026)",

```

```

    "url": "https://thenewpublishingstandard.com/2026/04/01/ani-vs-openai-delhi-high-court-judgment-ai-copyright-india/"
  },
  {
    "name": "DPDPA s.9 ? children's data: verifiable consent + behavioural-tracking/targeted-ads prohibition",
    "url": "https://www.dpdpa.com/dpdpa2023/chapter-2/section9.html"
  },
  {
    "name": "DPDP Rules 2025, Rule 10 ? verifiable parental consent",
    "url": "https://www.dpdpa.com/dpdparules/rule10.html"
  },
  {
    "name": "DPDP Rules 2025, Rule 12 ? exemptions from s.9(1)/s.9(3) for Fourth Schedule classes/purposes",
    "url": "https://www.dpdpa.com/dpdparules/rule12.html"
  },
  {
    "name": "Medianama ? New Data Rules Allow Child Tracking for Safety Without Consent (Rule 12 carve-out)",
    "url": "https://www.medianama.com/2025/11/223-dpdp-rules-real-time-child-tracking-without-consent/"
  },
  {
    "name": "CyberPeace ? Prohibition of behavioural tracking and targeted advertising for children under DPDP s.9(3)",
    "url": "https://cyberpeace.org/resources/blogs/prohibition-of-behavioral-tracking-and-targeted-advertising-for-children-under-the-dpdp-act-2023"
  },
  {
    "name": "S.S. Rana & Co. ? Protecting trade secrets in India (Contract Act 1872, breach of confidence, IT Act s.72)",
    "url": "https://ssrana.in/articles/protecting-trade-secrets-india/"
  },
  {
    "name": "K Singhania & Co. ? Regulation of biometric data under the DPDP Act (facial data = personal data)",
    "url": "https://ksandk.com/data-protection-and-data-privacy/regulation-of-biometric-data-under-the-dpdp-act/"
  },
  {
    "name": "Van Buren v. United States, No. 19-783 (U.S. June 3, 2021) ? slip opinion",
    "url": "https://www.supremecourt.gov/opinions/20pdf/19-783_k53l.pdf"
  },
  {
    "name": "Zwillgen ? Van Buren: footnote 8 reserved whether contract/policy limits ground CFAA liability",
    "url": "https://www.zwillgen.com/alternative-data/scotus-rules-cfaa-does-not-contain-purpose-based-restrictions/"
  },
  {
    "name": "Privacy World (Squire Patton Boggs) ? hiQ v. LinkedIn breach-of-contract summary judgment",
    "url": "https://www.privacyworld.blog/2022/11/federal-court-rules-in-favor-of-linkedins-breach-of-contract-claim-after-six-years-of-cfaa-data-scraping-litigation/"
  },
  {
    "name": "Privacy World ? hiQ v. LinkedIn $500k consent judgment, injunction and destruction of scraped-derived data",
    "url": "https://www.privacyworld.blog/2022/12/linkedin-data-scraping-battle-with-hiq-labs-ends-with-proposed-judgment/"
  },
  {
    "name": "Eric Goldman / Quinn Emanuel ? Meta v. Bright Data (N.D. Cal. Jan. 23, 2024): logged-off public scraping not barred",

```

```

    "url": "https://www.quinnemanuel.com/the-firm/news-events/client-alert-meta-v-bright-data-significant-decision-for-web-scraping-industry/"
  },
  {
    "name": "Crowell & Moring ? browsewrap vs clickwrap enforceability (notice + assent)",
    "url": "https://www.crowell.com/en/insights/client-alerts/recent-court-rulings-provide-warnings-on-the-use-of-browsewrap-agreements"
  },
  {
    "name": "Hunton ? sign-in wrap enforceability: courts deny absent conspicuous notice / unambiguous assent",
    "url": "https://www.hunton.com/privacy-and-information-security-law/its-a-wrap-the-latest-from-the-ninth-circuit-on-sign-in-wrap-agreements"
  },
  {
    "name": "YouTube Terms of Service ? Permissions and Restrictions (automated access; harvest faces; redistribution)",
    "url": "https://www.youtube.com/static?template=terms"
  },
  {
    "name": "18 U.S.C. § 1839 ? Trade Secret Definitions (Cornell LII)",
    "url": "https://www.law.cornell.edu/uscode/text/18/1839"
  },
  {
    "name": "Gowling WLG ? Trade secrets in professional sports",
    "url": "https://gowlingwlg.com/en/insights-resources/articles/2023/trade-secrets-in-professional-sports"
  },
  {
    "name": "Lexology ? In-venue filming of live sporting events: ticket as contract; venue/broadcaster rights",
    "url": "https://www.lexology.com/library/detail.aspx?g=09fe6573-daf6-4fff-bcf0-b9c6ccf41792"
  },
  {
    "name": "GDPR Recital 26 (official text, gdpr-info.eu)",
    "url": "https://gdpr-info.eu/recitals/no-26/"
  },
  {
    "name": "gdpr-info.eu ? Personal Data (Art.4(1)); WP136 'relates to' test",
    "url": "https://gdpr-info.eu/issues/personal-data/"
  },
  {
    "name": "gdpr-info.eu ? Art.9 GDPR (special-category biometric data); ICO biometric guidance",
    "url": "https://gdpr-info.eu/art-9-gdpr/"
  },
  {
    "name": "gdpr-info.eu ? Art.8 GDPR (children's consent)",
    "url": "https://gdpr-info.eu/art-8-gdpr/"
  },
  {
    "name": "Fox Rothschild ? EDPB Guidelines 3/2019 on video devices (Art.9 only on unique identification)",
    "url": "https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance"
  },
  {
    "name": "Weiss et al., Sensors 2025 (PMC) ? Privacy Beyond the Face: gait identity survives pose anonymization",
    "url": "https://pmc.ncbi.nlm.nih.gov/articles/PMC12788357/"
  },
  {
    "name": "Bird & Bird ? CJEU C-582/14 Breyer: relative approach to identifiability",
    "url": "https://www.twobirds.com/en/insights/2016/global/cjeu-decision-on-dynamic-ip-

```

```

addresses-touches-fundamental-dp-law-questions"
  },
  {
    "name": "Article 29 Working Party Opinion 05/2014 (WP216) on Anonymisation Techniques",
    "url": "https://ec.europa.eu/justice/article-29/documentation/opinion-recommendation/files/2014/wp216_en.pdf"
  },
  {
    "name": "Hunton ? EDPB Guidelines 01/2025 on pseudonymisation (context-dependent identifiability)",
    "url": "https://www.hunton.com/insights/publications/edpb-advises-on-pseudonymisation-for-gdpr-compliance"
  },
  {
    "name": "Clifford Chance ? Pseudonymized data after EDPS v SRB (CJEU C-413/23 P, 4 Sept 2025)",
    "url": "https://www.cliffordchance.com/insights/resources/blogs/talking-tech/en/articles/2025/09/pseudonymized-data-after-edps-v-srb.html"
  },
  {
    "name": "Skadden ? EU Court Clarifies When Pseudonymised Data Is Not Personal Data (EDPS v SRB)",
    "url": "https://www.skadden.com/insights/publications/2025/11/in-a-landmark-decision-eu-court-clarifies"
  },
  {
    "name": "ICO ? How do we ensure anonymisation is effective? (motivated-intruder test)",
    "url": "https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-sharing/anonymisation/how-do-we-ensure-anonymisation-is-effective/"
  },
  {
    "name": "NIST CSRC ? SP 800-188 De-Identifying Government Datasets (release-model framing)",
    "url": "https://csrc.nist.gov/News/2023/nist-publishes-sp-800-188"
  },
  {
    "name": "Sweeney, k-Anonymity: A Model for Protecting Privacy (EPIC mirror)",
    "url": "https://epic.org/wp-content/uploads/privacy/reidentification/Sweeney_Article.pdf"
  },
  {
    "name": "de Montjoye et al., Unique in the Crowd (Scientific Reports 2013) ? 4 points -> 95%",
    "url": "https://dspace.mit.edu/bitstream/handle/1721.1/88233/de%20Montjoye-2013-Unique%20in%20the%20crowd.pdf?sequence=1&isAllowed=y"
  },
  {
    "name": "OVIC ? The Limitations of De-Identification (data environment / external linkage; forward-looking)",
    "url": "https://ovic.vic.gov.au/privacy/resources-for-organisations/the-limitations-of-de-identification-protecting-unit-record-level-personal-information/"
  },
  {
    "name": "Trajectory-Based Spatiotemporal Entity Linking (arXiv 2010.01516)",
    "url": "https://arxiv.org/pdf/2010.01516"
  }
]
}

```

GROUNDING:

CODEBASE FACTS (verified by the lead before this run):

- eval_baselines/ is a TRACKED (committed) dir of JSON baseline snapshots (regression_baseline.json, heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json, experiment_leaderboard.json). Precedent: committed structured regression files already live in git.
- backend/eval/regression.py loads a committed baseline, scores DB-stored predictions vs ground-

truth intervals, flags P/R drop beyond tolerance, with incommensurability guards (IoU/detector/GT-hash). Predictions come from the DB (needs a pipeline run on the video) -> NOT video-free on the prediction side yet.

- backend/eval/fusion_golden.py extracts *_golden_features.json from VIDEO+trajectory CSV+labels (needs GPU/WSL). backend/eval/fusion_compare.py & backend/eval/ablation.py RE-SCORE those JSON features on pure CPU with NO video (LOVO deltaF1, bootstrap CI, sign test). This is the EXISTING VIDEO-FREE seam.
- docs/GOLDEN_DATA_SHARING.md (#175): *_golden_features.json are deliberately kept PRIVATE on OneDrive, OUT of git, because they derive from a 'Proprietary corpus.' Existing skipif litmus tests (tests/test_ablation.py::test_litmus_reproduces_gate_a, tests/test_fusion_compare.py) run ONLY if those features are present in output/ -> currently skipped everywhere.
- Data contracts: trajectory CSV columns (Frame,Visibility,X,Y); candidate/final rally segments = (start,end) time intervals; multi-signal fusion uses shuttle + person + audio + court-bounds feature columns (weak signals retained, default-OFF). video_id = MD5 of the source file (backend/utils/hashing.py) ? i.e. using it as a public key would FINGERPRINT the source file.
- Guardrails: commercial-clean licensing MIT/Apache/BSD only; paid LLMs never a training source; EXCLUDE MINORS from the training pool; gait/biometrics strictly FUTURE (GDPR Art 9). Owner is Bangalore/India-based; aims at commercialization and possibly co-authoring a research paper.
- A SEPARATE workflow is concurrently designing the general video-free fixture mechanics. THIS effort focuses on (a) the LEGAL/IP/privacy status of committing DERIVED structured data PUBLICLY, and (b) a non-attributability + encryption ('secret file') architecture.

GOAL: let an external contributor on another clone run the rally-segmentation regression suite from COMMITTED structured fixtures (no video present), such that the fixtures (i) cannot be attributed to a specific source video/match/person and (ii) are useless to a public user in isolation.

KEY CONSTRAINT ALREADY ESTABLISHED BY THE LEAD: a contributor who RUNS the tests necessarily can read the plaintext fixtures + any key on their machine (DRM/analog-hole limit). So secrecy-from-contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-isolation, plus encryption only as defense against the NO-KEY public.

2. assistant (2026-06-16T03:05:59.611Z)

I'll start by reading the existing infrastructure to ground the schema design in what's actually there, then sketch the protobuf fixture schema and harness integration.

3. user (2026-06-16T03:06:02.606Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-16T03:06:02.619Z)

```

1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7          ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8          predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9          comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10         baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23

```

```

24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                     f1: float, path: str = DEFAULT_BASELINE_PATH,
82                     iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                     gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a

```

```

87         later run computed under different settings (or against different labels) is
detected
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105           stage: str = "final", iou_threshold: float = 0.5,
106           detector: Optional[str] = None,
107           tolerance: float = DEFAULT_TOLERANCE,
108           baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:

```

```

143         regressed = True
144         reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "\n    - {why}" for why in r.reasons)
189

```

5. user (2026-06-16T03:06:02.621Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature

```



```

columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42         if spec.get("video_path"):
43             return str(spec["video_path"])
44         labels = str(spec["labels"])
45         if labels.endswith(".rallies.csv"):
46             return labels[: -len(".rallies.csv")] + ".MP4"
47         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                      frame_skip: int = 3, iou: float = 0.5,
52                      log_every_sec: float = 15.0,
53                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle{}}, person{}, label]}, gts}`.
57         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
58         when None a real `PersonDetector` is built (loads torchvision on first use).
59         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60         from the detection progress callback so a hard kill leaves a forensic runlog.
61         """
62         fps, fw = float(spec["fps"]), float(spec["frame_width"])

```

```

63     traj = str(spec["trajectory"])
64     points = TrackNetRunner.parse_trajectory_csv(traj)
65     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66     gts = load_golden_gts(str(spec["labels"]))
67     labels = label_candidates(intervals, gts, iou)
68     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70     axis, net_px, _span = rally_gate.estimate_play_axis(points)
71     video_path = _derive_video_path(spec)
72
73     if detector is None:
74         from backend.pipeline.detectors.person_detector import PersonDetector
75         detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79     n = len(intervals)
80     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                             court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {

```

```

123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
135             # last);
136             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
137             # row/frame).
138             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
139                                                if os.path.isfile(s.get("trajectory", ""))
140                                                else 0))
141         os.makedirs(out_dir, exist_ok=True)
142         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
143         # keep a tiny
144         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
145         # must never
146         # break extraction, so guard the build and fall back to no telemetry on any error.
147         telem: Optional[RunTelemetry]
148         try:
149             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
150                                label="fusion_golden")
151         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
152             # observes
153             telem = None
154         written: List[str] = []
155         for vi, spec in enumerate(specs, 1):
156             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
157             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
158             # prior
159             # run left off instead of re-detecting). --fresh forces a redo.
160             if not fresh and os.path.isfile(out):
161                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
162                 extracted "
163                     f"-> {out} (use --fresh to redo)", flush=True)
164                 written.append(out)
165                 continue
166             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
167                   flush=True)
168             t0 = time.monotonic()
169             if telem is not None:
170                 telem.event("video_start", spec["name"])
171             try:
172                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
173             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
174                 # raise
175                 if telem is not None:
176                     telem.event("error", spec["name"], err=str(e))
177                 raise
178             n_pos = sum(c["label"] for c in data["candidates"])
179             with open(out, "w", encoding="utf-8") as f:
180                 json.dump(data, f, indent=2)
181             if telem is not None:
182                 telem.event("video_done", spec["name"],
183                           windows=len(data["candidates"]), positives=n_pos)
184             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
185             {time.monotonic() - t0:.0f}s "
186                 f"| {len(data['candidates'])} windows ({n_pos} positive) |
187             {len(data['gts'])} golden rallies "

```

```

175         f"-> {out}", flush=True)
176         written.append(out)
177         return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

6. assistant (2026-06-16T03:06:06.241Z)

Now let me look at the video-free CPU re-scoring seam and the litmus tests, plus the distill_local feature module and the golden data sharing doc.

7. user (2026-06-16T03:06:07.192Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6     designed seam): the person features are extra columns the learned LogisticRegression
weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test

```

```

27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos.""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58         feature_names=list(SHUTTLE_FEATURE_NAMES),
59         threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61         feature_names=list(SHUTTLE_FEATURE_NAMES) +
62         list(PERSON_FEATURE_NAMES),
63         threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
72             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
73             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
74             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
75             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
76             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
77             f"| CI clears 0: {ds['ci_excludes_zero']}",
78             f"  per-video ?F1:",
79         ]
80         for pv in res["per_video_delta"]:
81             lines.append(f"    {pv['video']}<26s} ?F1={pv['f1']:+.3f}")
82         return "\n".join(lines)

```

```

83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

8. user (2026-06-16T03:06:08.053Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
4     (``scratch/gate_ab_ablation.py``):
5     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
6     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
7     95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
8     (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
9
10    **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
11    ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
12    ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
13
14    ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
primary
15
16    operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
17    ablated (smaller) variant, B is full, so the reported B-A delta IS ?(s), a per-video-
paired ?F1
18    with bootstrap CI + exact sign test, LOVO-honest for learned variants.
19
20    Two ablation granularities fall out of one inventory:
21
22    - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
23    - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
24
25    "is this old SIGNAL still relevant?" question at cue granularity).
26
27    A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
28    a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
29    dropping a column). The runner handles both uniformly: ``without_s`` subtracts the

```

```

columns AND/OR
28     zeroes the gate knob from ``cfg.full``.
29
30     Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; S6 of the
plan):
31     - never a bare mean ? every row ships a CI + the exact sign test;
32     - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33         agreeing** (the ``per_user_gate`` bar, in reverse);
34     - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35         the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36         flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37         but tags ``structural_protected``;
38     - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39         MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42     writes one ``<name>_golden_features.json`` per video). CLI::
43
44     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45     """
46     from __future__ import annotations
47
48     import argparse
49     import glob
50     import json
51     import logging
52     import os
53     from dataclasses import dataclass, field
54     from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56     from backend.eval import experiment
57     from backend.eval.experiment import Variant, VideoCandidates
58
59     logger = logging.getLogger(__name__)
60
61     # --- The signal inventory (the things the runner ablates), grouped by producer.
-----
62     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63     # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                         "active_ratio", "net_crossings")
67     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                         "in_court_ratio", "shuttle_player_colocation")
69     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72     EARNS_KEEP = "earns_keep"                # CI clears 0 AND sign test agrees ? removing it
hurts
73     SUGGESTIVE = "suggestive"                # point estimate leans positive but CI spans 0
74     INERT = "inert"                          # wash this run (?0, CI spans 0) ? candidate,
not proof
75     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
lift"

```

```

76
77 #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78 VERDICT_LABEL = {
79     EARNS_KEEP: "clears-bar",
80     SUGGESTIVE: "suggestive",
81     INERT: "inert",
82     STRUCTURAL_PROTECTED: "structural-protected",
83 }
84
85
86 @dataclass(frozen=True)
87 class Signal:
88     """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90     - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91     - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
whole group.
92     - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93     pure gate signal like Gate-A whose column is shared with the shuttle group).
94     - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95     the full variant rather than dropping a column.
96     - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97     flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98     """
99
100     name: str
101     group: str
102     columns: Tuple[str, ...] = ()
103     gate_knob: Optional[str] = None
104     structural: bool = False
105
106
107 @dataclass
108 class AblationConfig:
109     """How to run an ablation over a corpus.
110
111     - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112     the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113     ``feature_names`` = all columns and the gates ON.
114     - ``signals`` ? the inventory to ablate.
115     - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116     - ``iou`` ? temporal-IoU match threshold.
117     - ``honest`` ? LOVO for learned variants (the number to trust).
118     - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119     - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120     """
121
122     full: Variant
123     signals: List[Signal]
124     granularity: str = "group"
125     iou: float = 0.5
126     honest: bool = True
127     seed: int = 0
128     p_threshold: float = 0.05
129
130
131 @dataclass
132 class AblationRow:

```



```

133         """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
verdict)."""
134
135         signal: str
136         group: str
137         structural: bool
138         dropped_columns: List[str]
139         gate_knob: Optional[str]
140         n: int
141         delta_f1: float
142         ci_low: float
143         ci_high: float
144         ci_excludes_zero: bool
145         sign_wins: int
146         sign_losses: int
147         sign_p_value: float
148         f1_full: float
149         f1_ablated: float
150         seg_ratio_full: float
151         seg_ratio_ablated: float
152         verdict: str
153         structural_protected: bool
154
155         def as_dict(self) -> Dict[str, Any]:
156             return {
157                 "signal": self.signal,
158                 "group": self.group,
159                 "structural": self.structural,
160                 "dropped_columns": list(self.dropped_columns),
161                 "gate_knob": self.gate_knob,
162                 "n": self.n,
163                 "delta_f1": self.delta_f1,
164                 "ci_low": self.ci_low,
165                 "ci_high": self.ci_high,
166                 "ci_excludes_zero": self.ci_excludes_zero,
167                 "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                             "p_value": self.sign_p_value},
169                 "f1_full": self.f1_full,
170                 "f1_ablated": self.f1_ablated,
171                 "segment_count_ratio_full": self.seg_ratio_full,
172                 "segment_count_ratio_ablated": self.seg_ratio_ablated,
173                 "verdict": self.verdict,
174                 "structural_protected": self.structural_protected,
175             }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
result."""
181
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
order)."""
193             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
194

```

```

195     def as_dict(self) -> Dict[str, Any]:
196         return {
197             "granularity": self.granularity,
198             "iou": self.iou,
199             "num_videos": self.num_videos,
200             "honest_lovo": self.honest_lovo,
201             "full_set_signals": list(self.full_set_signals),
202             "full_spec_hash": self.full_spec_hash,
203             "label_set_hash": self.label_set_hash,
204             "rows": [r.as_dict() for r in self.sorted_rows()],
205         }
206
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
212         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
213         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214         mutates ``full``."""
215         cols = set(columns)
216         fn = full.feature_names
217         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219         if gate_knob == "min_crossings":
220             overrides["min_crossings"] = 0
221         elif gate_knob == "min_players":
222             overrides["min_players"] = 0
223         elif gate_knob is not None:
224             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227             "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231
232         sign_p: float, p_threshold: float) -> Tuple[str, bool]:
233         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
234
235         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
236         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
237         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
238         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
239         if structural:
240             return STRUCTURAL_PROTECTED, True
241         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
242         if ci_excludes_zero and sign_agrees:
243             return EARNS_KEEP, False
244         if sign_wins > sign_losses:
245             # leans toward "removal hurts" but evidence is
thin
246             return SUGGESTIVE, False
247         return INERT, False
248
249     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
250         signal: Signal) -> AblationRow:
251         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.

```

```

251
252 Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253 variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254 Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255 come for free.""
256 ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257 res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                          honest_stats=True)
259 # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260 # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261 from backend.eval.eval_stats import paired_delta_ci
262 by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263 by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264 vids = [p["video"] for p in res["variant_a"]["per_video"]]
265 pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266 st = pd["sign_test"]
267 sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268 sign_p = float(st["p_value"])
269 verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                               sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272 seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273 seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275 return AblationRow(
276     signal=signal.name,
277     group=signal.group,
278     structural=signal.structural,
279     dropped_columns=list(signal.columns),
280     gate_knob=signal.gate_knob,
281     n=int(pd["n"]),
282     delta_f1=float(pd["mean_delta"]),
283     ci_low=float(pd["ci_low"]),
284     ci_high=float(pd["ci_high"]),
285     ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286     sign_wins=sign_wins,
287     sign_losses=sign_losses,
288     sign_p_value=sign_p,
289     f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290     f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291     seg_ratio_full=float(seg_full),
292     seg_ratio_ablated=float(seg_abl),
293     verdict=verdict,
294     structural_protected=protected,
295 )
296
297
298 def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299     """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
300     OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301     order: List[str] = []
302     by_group: Dict[str, List[Signal]] = {}
303     for s in signals:
304         if s.group not in by_group:
305             order.append(s.group)
306             by_group[s.group] = []
307         by_group[s.group].append(s)
308     out: List[Signal] = []
309     for g in order:

```

```

310         members = by_group[g]
311         cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312         knob = next((m.gate_knob for m in members if m.gate_knob), None)
313         structural = any(m.structural for m in members)
314         out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315     return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ----- default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the

```

```

whole cue."""
359     return [
360         Signal("shuttle.duration", "shuttle", ("duration",)),
361         Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362         Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363         Signal("person", "person", PERSON_COLUMNS),
364         Signal("audio", "audio", AUDIO_COLUMNS),
365     ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (`scratch/gate_ab_ablation.py`)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395                 cands = d["candidates"]
396                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397                 features: Dict[str, List[float]] = {}
398                 for fn in SHUTTLE_COLUMNS:
399                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
400                 for fn in PERSON_COLUMNS:
401                     features[fn] = [float(c["person"][fn]) for c in cands]
402                 for fn in AUDIO_COLUMNS:
403                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404                 gts = [(float(a), float(b)) for a, b in d["gts"]]
405                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
features=features, gts=gts))
406
407         return videos
408
409
410     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:

```

```

411         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
412         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
413         (distinct from the static Gate-A litmus)."""
414         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
415         return experiment.Variant(name="full_learned", feature_names=cols,
416                                 threshold=threshold, min_crossings=min_crossings)
417
418
419     # ----- table
rendering
420
421
422     def format_table(report: AblationReport) -> str:
423         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
424         rows = report.sorted_rows()
425         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
426                 f" . n={report.num_videos} . IoU {report.iou}"
427                 f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
428                 f" . {reportgranularity.upper()}")
429         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
430         widths = [18, 17, 18, 14, 5, 22]
431
432         def _fmt_row(cells: Sequence[str]) -> str:
433             return " ".join(c.ljust(w) for c, w in zip(cells, widths))
434
435         lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
436         for r in rows:
437             ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
438             sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
439             verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
440             lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
441                                   "yes" if r.ci_excludes_zero else "no", verdict]))
442         return "\n".join(lines)
443
444
445     # ----- PDF
export
446
447
448     def export_pdf(report: AblationReport, path: str) -> str:
449         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
marginal
450         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
451         structural-protected). Reuses the muted booktabs aesthetic of
``scratch/quality_report.py``.
452
453         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
GPU)."""
454         from reportlab.lib import colors
455         from reportlab.lib.enums import TA_CENTER, TA_LEFT
456         from reportlab.lib.pagesizes import A4
457         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
458         from reportlab.lib.units import cm
459         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle)
460
461         navy = colors.HexColor("#1F3A5F")
462         black = colors.HexColor("#000000")
463         darkgrey = colors.HexColor("#333333")
464         light = colors.HexColor("#F0F0F0")
465         hairline = colors.HexColor("#BFBFBF")
466         row_alt = colors.HexColor("#F7F7F7")

```

```

467
468     base = getSampleStyleSheet()["Normal"]
469     title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
fontSize=16,
470                             leading=20, alignment=TA_CENTER, textColor=navy,
spaceAfter=4)
471     subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
fontSize=9.5,
472                             leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
473     h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
leading=15,
474                             textColor=navy, spaceBefore=10, spaceAfter=4)
475     body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
leading=12.5,
476                             alignment=TA_LEFT, textColor=black, spaceAfter=5)
477     cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
478                             textColor=black)
479     cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
480                             leading=10.5, textColor=black)
481     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482                             leading=10.5, textColor=black)
483
484     rows = report.sorted_rows()
485     story: List[Any] = [
486         Paragraph("Signal Ablation & Feature-Enablement Report", title),
487         Paragraph(
488             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489             f"contribution & n={report.num_videos} golden matches, IoU
{report.iou}, "
490             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491             "Honest measured results only.",
492             subtitle),
493     ]
494
495     full_set = ", ".join(report.full_set_signals)
496     story.append(Paragraph("Run provenance", h2))
497     story.append(Paragraph(
498         f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499         f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
500         f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
501         "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
502         "bootstrap 95% CI and an exact paired sign test &dash; a signal earns its keep
only "
503         "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
504         body))
505
506     story.append(Paragraph("Marginal contribution per signal", h2))
507     table_data: List[List[Any]] = [[
508         Paragraph("Signal", head), Paragraph("Group", head),
509         Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
510         Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
511         Paragraph("Over-seg", head), Paragraph("Verdict", head),
512     ]]
513     for r in rows:
514         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
515         sign = f"{r.sign_wins}/{r.sign_losses}L p={r.sign_p_value:.3f}"
516         overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
517         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)

```

```

518         df1_style = cell_b if r.verdict == EARNNS_KEEP else cell
519         table_data.append([
520             Paragraph(r.signal, cell), Paragraph(r.group, cell),
521             Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
522             Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
523             Paragraph(overseg, cell), Paragraph(verdict, cell),
524         ])
525     col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
526     tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
527     last = len(table_data) - 1
528     tbl.setStyle(TableStyle([
529         ("BACKGROUND", (0, 0), (-1, 0), light),
530         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
531         ("TOPPADDING", (0, 0), (-1, -1), 4),
532         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
533         ("LEFTPADDING", (0, 0), (-1, -1), 4),
534         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
535         ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
536         ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
537         ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
538         ("LINEBELOW", (0, last), (-1, last), 1.0, black),
539         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
540     ]))
541     story.append(tbl)
542     story.append(Spacer(1, 0.3 * cm))
543
544     story.append(Paragraph("Verdict key", h2))
545     story.append(Paragraph(
546         "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
547         "and the sign test agreeing (the signal earns its keep). "
548         "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
549         "spans 0 (keep, watch as the corpus grows). "
550         "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
551         "trend across multiple runs, never on one wash at small n. "
552         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
553         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
554         "never auto-flagged dead weight on 'no measured lift'.",
555         body))
556
557     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
558         topMargin=1.8 * cm, bottomMargin=1.8 * cm,
559         title="Signal Ablation & Feature-Enablement Report",
560         author="badminton-highlight-indexer")
561     doc.build(story)
562     logger.info("wrote ablation PDF -> %s", path)
563     return path
564
565
566 # -----
CLI
567
568
569 def main() -> None:
570     ap = argparse.ArgumentParser(
571         description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
572         "over experiment.compare + eval_stats; no new scoring math).")
573     ap.add_argument("--features-dir", default="output",

```



```

574             help="dir of *_golden_features.json (the persisted golden
substrate)")
575         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
576             help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
577         ap.add_argument("--iou", type=float, default=0.5)
578         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
579         ap.add_argument("--litmus", action="store_true",
580             help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
581                 "instead of the learned full-set ablation")
582         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
583         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
584         args = ap.parse_args()
585
586         videos = load_videos(args.features_dir)
587         if len(videos) < 2:
588             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
589                 f"(found {len(videos)})")
590
591         if args.litmus:
592             cfg = litmus_config()
593             cfg.iou, cfg.seed = args.iou, args.seed
594         else:
595             cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
596                 granularity=args.granularity, iou=args.iou, seed=args.seed)
597
598         report = run_ablation(videos, cfg)
599         print(format_table(report))
600
601         if args.json:
602             with open(args.json, "w", encoding="utf-8") as f:
603                 json.dump(report.as_dict(), f, indent=2)
604             print(f"\nwrote report JSON -> {args.json}")
605         if args.pdf:
606             export_pdf(report, args.pdf)
607             print(f"wrote PDF report -> {args.pdf}")
608
609
610         if __name__ == "__main__":
611             main()
612

```

9. user (2026-06-16T03:06:08.302Z)

```

1         # Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions
2
3         **Status:** convention + plan (2026-06-15). Tracked in ##175.
4         **Purpose:** a single cloud-synced location for the GPU-extracted golden artifacts
(trajjectory CSVs, `*_golden_features.json`, manifests, baseline snapshots) so the **GPU box**
and **CPU dev/agent sessions** share the same files ? without committing large data to git.
5
6         ## Why this exists
7         The rally-quality measurement is split across two machines by capability:
8         - **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the
**videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL**
stack ? only the GPU box can run it.
9         - **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO
**litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the
small **_golden_features.json` ? any session (incl. a CPU dev container) can run them.
10
11         Today those artifacts sit in gitignored `output/` (per-machine) and scattered local
folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs

```

them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap with OneDrive.

```
12
13     ## Canonical shared folder
14     `%USERPROFILE%\OneDrive\rally-annotator\golden-data\` ?
15     `C:\Users\avidu\OneDrive\rally-annotator\golden-data\`
16     OneDrive is present + signed-in on both the GPU box and the dev machine under the same
17     account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an
18     overwrite):
19     ```
20     golden-data/
21     README.md
22     features/<corpus-tag>/      *_golden_features.json      (small ? the key shared
23     artifact)
24     trajectories/<corpus-tag>/  *_predict.csv / *_wasb.csv (medium ? reusable across
25     runs)
26     manifests/                  *_lovo_manifest.json
27     baselines/                  leaderboard / regression snapshots (or keep tracked in
28     eval_baselines/)
29     ```
30     `<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos
31     here (tens of GB) ? keep those local to the GPU box.
32
33     ## The GPU ? CPU workflow
34     1. **GPU box** (videos + CSVs + labels present):
35     ```
36     python -m backend.eval.fusion_golden --manifest <manifest> \
37         --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
38     data\features\2026-06-15-n6"
39     ```
40     (or extract to local `output/` then copy the `*_golden_features.json` into the shared
41     folder).
42     2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
43     3. **CPU dev/agent session** points the analysis at the shared folder:
44     ```
45     python -m backend.eval.fusion_compare --features-dir "...golden-
46     data\features\2026-06-15-n6" --record
47     python -m backend.eval.ablation      --features-dir "...golden-
48     data\features\2026-06-15-n6" --granularity group
49     ```
50     ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/`
51     to also light up the `skipif` golden-litmus tests
52     (`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).
53
54     ## Optional code ergonomics (in #175, not required)
55     - A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` /
56     `fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` /
57     `--features-dir` flags already make this a convention; the env var is just ergonomics.)
58     - A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared
59     folder.
60
61     ## Guardrails
62     - The shared folder lives **outside the repo** ? never committed. Keep `output/`
63     gitignored and do **not** symlink the shared folder into a tracked path.
64     - The features derive from the **Proprietary** corpus ? keep the OneDrive folder
65     **private** (no public share links).
66
67     See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor),
68     `docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector
69     setup).
```

10. assistant (2026-06-16T03:06:11.013Z)

Now let me look at the `experiment.py` VideoCandidates/Variant structures, the distill_local feature names, the audio features, and the existing litmus tests so I can be precise about what to extend.

11. user (2026-06-16T03:06:12.356Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. **No new scoring math lives
7      here**; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56     ratio
```

```

scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the control behaviour (no gate, no learned
82         filter), so a variant that merely carries a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105        """
106
107        name: str
108        segmenter: str = "wasb_hybrid"
109        config_overrides: Dict[str, Any] = field(default_factory=dict)
110        cue_flags: Dict[str, bool] = field(default_factory=dict)
111        min_crossings: int = 0
112        min_players: int = 0
113        classifier_model: Optional[str] = None
114        feature_names: Optional[List[str]] = None
115        threshold: float = 0.5
116        merge_gap: float = 1.0
117        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118        # point is chosen on the TRAINING fold, never the held-out one ? fixing the

```

```

first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False             # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None           # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
125             return self.classifier_model is not None or bool(self.feature_names)
126
127         def to_dict(self) -> dict:
128             return {
129                 "name": self.name,
130                 "segmenter": self.segmenter,
131                 "config_overrides": self.config_overrides,
132                 "cue_flags": self.cue_flags,
133                 "min_crossings": self.min_crossings,
134                 "min_players": self.min_players,
135                 "classifier_model": self.classifier_model,
136                 "feature_names": self.feature_names,
137                 "threshold": self.threshold,
138                 "merge_gap": self.merge_gap,
139                 "tune_threshold": self.tune_threshold,
140                 "calibrate": self.calibrate,
141             }
142
143         @classmethod
144         def from_dict(cls, d: dict) -> "Variant":
145             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146             return cls(**{k: v for k, v in d.items() if k in known})
147
148         def spec_hash(self) -> str:
149             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150             and makes a result reproducible. The pinned **control** variant always hashes
the
151             same, so a regression is always traceable to a recipe change, never to drift."""
152             blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153             return sha1(blob.encode()).hexdigest()[:12]
154
155
156         #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157         #: filter. Always the comparison reference; "rollback" = make this the served variant.
158         CONTROL = Variant(name="control")
159
160
161         # ----- persisted candidates
162
163
164         @dataclass
165         class VideoCandidates:
166             """One video's persisted detection, ready for offline re-scoring.
167
168             ``intervals`` are the candidate windows; ``features`` is column-major
169             (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170             ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171             with ``load_video_candidates``, or directly in tests.
172             """
173
174             name: str
175             intervals: List[Interval]
176             features: Dict[str, List[float]] = field(default_factory=dict)
177             gts: Optional[List[Interval]] = None
178
179

```

```

180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213 def predict(variant: Variant, vc: VideoCandidates,
214            model: Optional[LogisticRegression] = None,
215            threshold: Optional[float] = None,
216            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226
227     """
228     mask = _gate_mask(variant, vc)
229     if variant.feature_names:
230         if model is None:
231             if variant.classifier_model is None:
232                 raise ExperimentError(
233                     f"variant {variant.name!r} is learned but has no model to predict "
234                     f"with (pass a fitted model or set classifier_model)")
235             model = LogisticRegression.load(variant.classifier_model)
236         proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238         if calibrator is not None:
239             proba = [calibrator(p) for p in proba]
240         thr = variant.threshold if threshold is None else threshold
241         mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242     kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243     return merge_overlaps(kept, gap_tol=variant.merge_gap)

```

```

241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                             train_videos: Sequence[VideoCandidates], iou: float
245                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                                                                 [])))
258                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259             if variant.calibrate == "platt":
260                 calibrator = fit_platt(raw, y)
261             elif variant.calibrate == "isotonic":
262                 calibrator = fit_isotonic(raw, y)
263             else:
264                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265                                     'platt'/'isotonic')")
266             threshold: Optional[float] = None
267             if variant.tune_threshold:
268                 best_t, best_f1 = variant.threshold, -1.0
269                 for k in range(1, 20):
270                     # grid 0.05..0.95 on the train
271                     t = round(0.05 * k, 2)
272                     f1s = [score(predict(variant, vc, model=model, threshold=t,
273                                         calibrator=calibrator),
274                               vc.gts or [], iou).f1 for vc in train_videos]
275                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276                     if mean_f1 > best_f1:
277                         best_f1, best_t = mean_f1, t
278                     threshold = best_t
279             return calibrator, threshold
280
281 # ----- variant scoring
282
283     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
284         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
285         the
286         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
287         X: List[List[float]] = []
288         y: List[int] = []
289         for vc in videos:
290             if vc.gts is None:
291                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
292             X.extend(_feature_matrix(vc, variant.feature_names or []))
293             y.extend(label_candidates(vc.intervals, vc.gts, iou))
294         if not X:
295             raise ExperimentError("cannot train a learned variant on zero candidates")
296         return LogisticRegression().fit(X, y, variant.feature_names)
297
298     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
299                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
300         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a

```

```

299         mean aggregate.
300
301     Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:
331                     raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332                 model: Optional[LogisticRegression] = _train(variant, others, iou)
333                 if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                     assert model is not None # _train never returns
None
335                     cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                      for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,

```



```

353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
361         mean)."""
362         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
363         _METRICS}
364
365     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
366     Dict[str, Any]:
367         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
368         hides)."""
369         gts_by_name = {v.name: (v.gts or []) for v in videos}
370         overlaps = _segm.DEFAULT_OVERLAPS
371         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
372         ratios: List[float] = []
373         for name, preds in eval_v.get("predictions", {}).items():
374             gts = gts_by_name.get(name, [])
375             got = _segm.f1_at_overlaps(preds, gts, overlaps)
376             for ov in overlaps:
377                 f1k[ov].append(got[ov])
378             ratios.append(_segm.segment_count_ratio(preds, gts))
379
380     def _mean(xs: List[float]) -> float:
381         return sum(xs) / len(xs) if xs else 0.0
382     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
383     f1k.items()}
384     out["segment_count_ratio"] = _mean(ratios)
385     return out
386
387
388     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
389     eval_b: Dict[str, Any]) -> Dict[str, Any]:
390         """C1 + C4 honest reporting layered over a compare() pair (additive,
391         EXPERIMENT_HARNESS §6).
392
393         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
394         order),
395         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
396         when
397         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
398         """
399         a_f1 = [p["f1"] for p in eval_a["per_video"]]
400         b_f1 = [p["f1"] for p in eval_b["per_video"]]
401         return {
402             "variant_a_ci": _ci_block(eval_a),
403             "variant_b_ci": _ci_block(eval_b),
404             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
405             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
406                             "variant_b": _segmentation_block(videos, eval_b)},
407         }
408
409
410     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
411     iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
412     Dict[str, Any]:
413         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
414         same
415         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
416
417

```

```

408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         ""
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result
449
450
451         # ----- SxS on unlabeled video
452
453
454         def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
agree_iou: float = 0.5) -> Dict[str, Any]:
455             """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
456
457             With no labels there is no lift to measure ? instead this surfaces where the two
458             variants **agree vs diverge**, which is exactly what a human reviews (and what two
459             highlight reels would show side by side):
460
461             - ``agreed`` ? windows both variants found (matched at ``agree_iou``);

```

```

463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465       the human / a later golden label adjudicates).
466
467 Both variants must be predictable without labels: a learned variant therefore needs
468 a
469 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470 `metrics.match_intervals` ? no new matching logic.
471 """
472 preds_a = predict(variant_a, video)
473 preds_b = predict(variant_b, video)
474 mr = match_intervals(preds_a, preds_b, agree_iou)
475
476 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477           for m in mr.matches]
478 agree_iou = [m.iou for m in mr.matches]
479 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482 def _dur(ivs: List[Interval]) -> float:
483     return sum(e - s for s, e in ivs)
484
485 return {
486     "video": video.name,
487     "agree_iou": agree_iou,
488     "variant_a": variant_a.name,
489     "variant_b": variant_b.name,
490     "num_a": len(preds_a),
491     "num_b": len(preds_b),
492     "num_agreed": len(agreed),
493     "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
494     "duration_a": _dur(preds_a),
495     "duration_b": _dur(preds_b),
496     "agreed": agreed,
497     "a_only": a_only,
498     "b_only": b_only,
499 }
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).

```

```

526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
538               row["variant_a"]["name"], row["variant_b"]["name"],
539               (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558         rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563         rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566         rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568         gts=gts)
569
570

```

12. user (2026-06-16T03:06:12.393Z)

```

1     """GPU-free tests for the LOSO/LOGO ablation runner (``backend.eval.ablation``).
2
3     Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5     - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
6     signal is
7     *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
8     zeroes
9     the gate knob without mutating ``full``.
10    - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
11    reported
12    (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
13    lift".
14    - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
15    stronger
16    than the stats support (the no-over-claim contract).
17    - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).

```

```

13     - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14     reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15     cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16     """
17     from __future__ import annotations
18
19     import glob
20     import json
21     import os
22
23     import pytest
24
25     from backend.eval import ablation
26     from backend.eval.ablation import (
27         AblationConfig,
28         AblationReport,
29         Signal,
30         ablate_one,
31         run_ablation,
32     )
33     from backend.eval.experiment import Variant, VideoCandidates
34
35
36     # ----- synthetic
fixtures
37
38
39     def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40         """``n`` videos with a ``net_crossings`` gate signal.
41
42         ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43         perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44         has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45         videos: list[VideoCandidates] = []
46         for k in range(n):
47             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48             junk_val = 5.0 if not separating else 0.0
49             feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50             gts = [(0.0, 10.0), (20.0, 30.0)]
51             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52         return videos
53
54
55     def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56         """``n`` videos where a PERSON feature (``players_in_court``) cleanly separates
rallies from
57         junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58         ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59         LOGO/LOSO correctness fixture (mirrors ``test_fusion_compare``'s construction)."""
60         videos: list[VideoCandidates] = []
61         for k in range(n):
62             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63             feats: dict[str, list[float]] = {}
64             for fn in ablation.SHUTTLE_COLUMNS:
65                 feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-
separating
66             for fn in ablation.PERSON_COLUMNS:

```

```

67         feats[fn] = [0.0, 0.0, 0.0, 0.0]
68         feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0]           # the separating signal
69         feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70         for fn in ablation.AUDIO_COLUMNS:
71             feats[fn] = [0.0, 0.0, 0.0, 0.0]
72         gts = [(0.0, 10.0), (20.0, 30.0)]
73         videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74     return videos
75
76
77     # ----- LOSO
correctness
78
79
80     def test_separating_gate_earns_keep():
81         """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82         drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83         videos = _gate_videos(6, separating=True)
84         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
86                             granularity="signal")
87         row = ablate_one(videos, cfg, sig)
88         assert row.delta_f1 > 0.0
89         assert row.ci_excludes_zero is True
90         assert row.sign_wins == 6 and row.sign_losses == 0
91         assert row.sign_p_value <= 0.05
92         assert row.verdict == ablation.EARNS_KEEP
93         assert row.structural_protected is False
94         # C4: the gate cuts over-segmentation (more junk segments without it).
95         assert row.seg_ratio_full < row.seg_ratio_ablated
96
97
98     def test_noop_gate_is_inert():
99         """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
104                             granularity="signal")
105         row = ablate_one(videos, cfg, sig)
106         assert abs(row.delta_f1) < 1e-9
107         assert row.ci_excludes_zero is False
108         assert row.verdict == ablation.INERT
109
110
111     def test_ablated_variant_does_not_mutate_full():
112         """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
113         full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
114         before = full.to_dict()
115         ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
116         assert ablated.feature_names == ["a", "c"]
117         assert ablated.min_crossings == 0
118         assert full.to_dict() == before           # full untouched
119         assert full.feature_names == ["a", "b", "c"]
120
121
122     def test_unknown_gate_knob_raises():
123         full = Variant(name="full", feature_names=["a"])
124         with pytest.raises(ValueError):
125             ablation._ablated_variant(full, (), "min_bogus")

```

```

126
127
128 # ----- LOGO
correctness
129
130
131 def test_logo_collapses_groups():
132     """LOGO (`granularity='group'`) ablates a whole producer's columns at once: the
default
133     5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
dropped."""
134     groups = ablation._collapse_to_groups(ablation.default_signals())
135     names = [g.name for g in groups]
136     assert names == ["shuttle", "gate_a", "person", "audio"]
137     shuttle = next(g for g in groups if g.name == "shuttle")
138     # the shuttle group unions duration + the velocity block
139     assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
"active_ratio"}
140     gate = next(g for g in groups if g.name == "gate_a")
141     assert gate.gate_knob == "min_crossings"
142     assert gate.structural is True # structural flag OR-ed up
143
144
145 def test_logo_finds_separating_group():
146     """On the learned fixture where PERSON cleanly separates, the LOGO person row must
be the
147     load-bearing one (largest positive marginal ?F1) and the constant groups must be
~washes."""
148     videos = _learned_videos(6)
149     cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
150     granularity="group")
151     report = run_ablation(videos, cfg)
152     rows = {r.signal: r for r in report.rows}
153     assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
154     top = report.sorted_rows()[0]
155     assert top.signal == "person"
156     assert rows["person"].delta_f1 > 0.0
157     # the constant audio group carries no information ? marginal ? 0
158     assert abs(rows["audio"].delta_f1) < 0.05
159
160
161 def test_run_ablation_both_granularities():
162     """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a
learned
163     full set."""
164     videos = _learned_videos(6)
165     loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
166     granularity="signal"))
167     logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
168     granularity="group"))
169
170     assert len(loso.rows) == len(ablation.default_signals()) # 5 LOSO rows
171     assert len(logo.rows) == 4 # 4 producer groups
172     assert loso.honest_lovo is True and logo.honest_lovo is True
173
174
175
176 def test_run_ablation_rejects_bad_granularity():
177     videos = _learned_videos(2)
178     cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
179     granularity="bogus")
180     with pytest.raises(ValueError):
181         run_ablation(videos, cfg)

```

```

182
183
184     def test_run_ablation_needs_a_video():
185         cfg = AblationConfig(full=ablation.learned_full_variant(),
186                               signals=ablation.default_signals())
187         from backend.eval.experiment import ExperimentError
188         with pytest.raises(ExperimentError):
189             run_ablation([], cfg)
190
191     # ----- structural-guard
192     exemption
193
194     def test_structural_guard_never_demoted():
195         """A structural guard with the SAME measured ? as a non-structural one that EARNs
196         its keep
197         must report verdict ``structural_protected`` (never auto-demoted), but record the
198         honest ?."""
199         videos = _gate_videos(6, separating=True)
200         non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
201         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
202         full = Variant(name="full", min_crossings=2)
203         cfg = AblationConfig(full=full, signals=[non], granularity="signal")
204         r_non = ablate_one(videos, cfg, non)
205         r_struct = ablate_one(videos, cfg, struct)
206         # identical honest measurement?
207         assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
208         assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins,
209         r_struct.sign_losses)
210         # ?but the verdict differs: non-structural earns keep, structural is protected.
211         assert r_non.verdict == ablation.EARNs_KEEP
212         assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
213         assert r_struct.structural_protected is True
214
215     def test_structural_guard_protected_even_when_inert():
216         """A structural guard that is a measured wash is STILL reported as
217         ``structural_protected``
218         (never flagged dead weight on 'no measured lift')."""
219         videos = _gate_videos(6, separating=False)
220         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
221         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
222                               granularity="signal")
223         row = ablate_one(videos, cfg, struct)
224         assert abs(row.delta_f1) < 1e-9 # honestly a wash
225         assert row.verdict == ablation.STRUCTURAL_PROTECTED
226         assert row.structural_protected is True
227
228     def test_default_gate_a_is_structural():
229         """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP
230         guard)."""
231         sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
232         assert sig.structural is True
233         assert sig.gate_knob == "min_crossings"
234
235     # ----- honest-
236     stats fields
237
238     def test_row_carries_full_honest_evidence():
239         """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict
240         in the

```



```

238     closed_vocabulary (no over-claim)."""
239     videos = _learned_videos(6)
240     cfg = AblationConfig(full=ablation.learned_full_variant(),
241                          signals=ablation.default_signals(),
242                          granularity="group")
243     report = run_ablation(videos, cfg)
244     valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
245             ablation.STRUCTURAL_PROTECTED}
246     for r in report.rows:
247         d = r.as_dict()
248         assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
249                "verdict"} <= set(d)
250         assert d["n"] == 6
251         assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
252         # CI brackets the mean ( $\pm$ fp) ? never a bare mean detached from its interval.
253         assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
254         assert r.verdict in valid
255
256     def test_classify_verdicts_cover_all_branches():
257         """`_classify` maps evidence ? the closed verdict vocabulary, honestly:
258         structural always protected; CI-clears-0 + sign-agrees ? earns_keep; leans-positive
259         but
260         CI-spans-0 ? suggestive; a wash ? inert."""
261         # structural always protected, regardless of evidence
262         assert ablation._classify(True, True, 6, 0, 0.01, 0.05) ==
263             (ablation.STRUCTURAL_PROTECTED, True)
264         assert ablation._classify(True, False, 0, 6, 1.0, 0.05) ==
265             (ablation.STRUCTURAL_PROTECTED, True)
266         # earns keep: CI clears 0 AND sign agrees
267         assert ablation._classify(False, True, 6, 0, 0.03, 0.05) == (ablation.EARNS_KEEP,
268             False)
269         # suggestive: leans positive (more wins) but CI spans 0 / not significant
270         assert ablation._classify(False, False, 4, 2, 0.30, 0.05) == (ablation.SUGGESTIVE,
271             False)
272         # inert: not more wins than losses and CI spans 0
273         assert ablation._classify(False, False, 2, 4, 0.69, 0.05) == (ablation.INERT, False)
274         assert ablation._classify(False, False, 2, 2, 1.0, 0.05) == (ablation.INERT, False)
275
276     def test_no_over_claim_vocabulary():
277         """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution
278         contract)."""
279         forbidden = {"irrelevant", "useless", "dead", "remove"}
280         for v in ablation.VERDICT_LABEL.values():
281             assert not any(bad in v.lower() for bad in forbidden)
282
283     def test_report_round_trips_json():
284         """The structured report is JSON-able and sorted by ?F1 desc (contribution
285         ranking)."""
286         videos = _learned_videos(6)
287         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
288             signals=ablation.default_signals(),
289             granularity="group"))
290         blob = json.dumps(report.as_dict())
291         back = json.loads(blob)
292         # must not raise
293         deltas = [r["delta_f1"] for r in back["rows"]]
294         assert deltas == sorted(deltas, reverse=True)
295         # sorted desc
296         assert back["num_videos"] == 6
297         assert back["label_set_hash"]
298         # provenance present
299
300     def test_seed_is_deterministic():
301         """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""

```

```

295     videos = _learned_videos(6)
296     cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
297         granularity="group", seed=7)
298     r1 = run_ablation(videos, cfg)
299     r2 = run_ablation(videos, cfg)
300     assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301     assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302
303
304     # ----- table +
PDF
305
306
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
310             signals=ablation.default_signals(),
311             granularity="group"))
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320
321
322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
328             signals=ablation.default_signals(),
329             granularity="group"))
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000 # non-trivial
334         reader = pypdf.PdfReader(out)
335         assert len(reader.pages) > 0
336         text = "".join(p.extract_text() for p in reader.pages)
337         assert "Ablation" in text
338         assert "clears-bar" in text # the verdict key rendered
339
340
341     # ----- corpus loader
(no GPU)
342
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
346         0.0 (loader never crashes on a pre-audio JSON)."""
347         cand = {"start": 0.0, "end": 10.0,
348             "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349             "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350             "label": 1}
351         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352             "candidates": [cand], "gts": [[0.0, 10.0]]}
353         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354             json.dump(data, f)

```

```

355     videos = ablation.load_videos(str(tmp_path))
356     assert len(videos) == 1
357     assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not
crashed
358
359
360     # ----- the
golden litmus
361
362     GOLDEN_DIR = os.path.join(
363         os.path.dirname(os.path.abspath(__file__)), "output")
364
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
370     @pytest.mark.skipif(not _golden_present(),
371         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
372     def test_litmus_reproduces_gate_a():
373         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
374         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
375         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
376         where the golden substrate is absent; runs on the owner box where it is present."""
377         videos = ablation.load_videos(GOLDEN_DIR)
378         assert len(videos) == 6
379         cfg = ablation.litmus_config()
380         report = run_ablation(videos, cfg)
381         assert len(report.rows) == 1
382         row = report.rows[0]
383         assert row.signal == "gate_a"
384         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
385         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
386         assert row.sign_wins == 6 and row.sign_losses == 0
387         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
388         assert row.ci_low == pytest.approx(0.042, abs=0.01)
389         assert row.ci_high == pytest.approx(0.104, abs=0.01)
390         assert row.ci_excludes_zero is True
391         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
392         assert row.verdict == ablation.STRUCTURAL_PROTECTED
393         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
394         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
395         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
396
397
398     @pytest.mark.skipif(not _golden_present(),
399         reason="golden output/*_golden_features.json absent")
400     def test_litmus_report_is_well_formed():
401         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
402         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
403         assert isinstance(report, AblationReport)
404         assert report.num_videos == 6
405         assert report.full_set_signals == ["gate_a"]
406         assert report.label_set_hash
407         out = ablation.format_table(report)
408         assert "gate_a" in out
409
410
411     # ----- edge
cases

```

```

412
413
414 def test_min_players_gate_knob_zeroed():
415     """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
416     full = Variant(name="full", min_players=2, feature_names=["a"])
417     ablated = ablation._ablated_variant(full, (), "min_players")
418     assert ablated.min_players == 0
419     assert ablated.feature_names == ["a"]          # columns untouched
420
421
422 def test_ablated_label_falls_back_to_gate_knob():
423     """With no columns, the ablated variant name carries the gate-knob label, not
424     '?'. """
425     full = Variant(name="full", min_crossings=2)
426     ablated = ablation._ablated_variant(full, (), "min_crossings")
427     assert "min_crossings" in ablated.name
428
429 # -----
430 CLI
431
432 def _write_learned_corpus(d: str, n: int = 6) -> None:
433     for k in range(n):
434         cand = []
435         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
436                             ((50.0, 51.0), 0)]:
437             shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
438             person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
439             person["players_in_court"] = 4.0 if pos else 0.0
440             person["in_court_ratio"] = 1.0 if pos else 0.0
441             audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
442             cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
443                         "audio": audio, "label": pos})
444         data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
445                "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
446         with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
447         as f:
448             json.dump(data, f)
449
450 def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
451     """``main()`` runs the LOGO ablation and writes both a PDF and the structured
452     JSON. """
453     pytest.importorskip("reportlab")
454     pytest.importorskip("pypdf")
455     d = str(tmp_path / "feat")
456     os.makedirs(d)
457     _write_learned_corpus(d)
458     pdf = str(tmp_path / "out.pdf")
459     js = str(tmp_path / "out.json")
460     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
461                                     "group",
462                                     "--pdf", pdf, "--json", js])
463     ablation.main()
464     out = capsys.readouterr().out
465     assert "Ablation report" in out
466     assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
467     assert os.path.isfile(js)
468     report = json.loads(open(js, encoding="utf-8").read())
469     assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
470 "audio"}
471
472 def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):

```

```

470     """`--litmus` runs the static Gate-A config (single structural row)."""
471     d = str(tmp_path / "feat")
472     os.makedirs(d)
473     _write_learned_corpus(d)
474     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
475     ablation.main()
476     out = capsys.readouterr().out
477     assert "gate_a" in out
478     assert "static" in out                                # litmus full set is the static
gate variant
479
480
481     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
482         d = str(tmp_path / "feat")
483         os.makedirs(d)
484         _write_learned_corpus(d, n=1)
485         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
486         with pytest.raises(SystemExit):
487             ablation.main()
488

```

13. assistant (2026-06-16T03:06:16.012Z)

I now have the full picture of the existing infra. Let me check the distill_local feature names and the trajectory parser to nail the exact data contract for the proto schema.

14. user (2026-06-16T03:06:17.703Z)

```

backend\pipeline\detectors\audio_features.py:15:AUDIO_FEATURE_NAMES = ["audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean"]
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\audio_features.py-16-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\audio_features.py-17-
backend\pipeline\detectors\audio_features.py-18-def compute_audio_features(hits: List,
window_start: float, window_end: float) -> Dict[str, float]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\audio_features.py-19-     """Audio features for one window
from a video's HitEvents (each has ``.t``, ``.strength``).
--
backend\pipeline\detectors\fusion_features.py:194:PERSON_FEATURE_NAMES = [
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-195-     "players_in_court",
"two_sided_motion", "mean_player_motion",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-196-     "in_court_ratio",
"shuttle_player_colocation",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-197-]
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\fusion_features.py-198-
--
backend\eval\distill_local.py:41:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
backend\eval\distill_local.py-42-# Recall-oriented proposal: deliberately permissive so real
rallies are among the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-43-#
candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-44-# Single-
sourced from backend.eval.windowing so eval and serve can never drift ? see that
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-45-# module's
docstring for the eval?serve bug these named presets close. The bare tuples are
--
backend\pipeline\detectors\tracknet_runner.py:237:     def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:

```

```

backend\pipeline\detectors\tracknet_runner.py-238-        """Parse a TrackNet predict CSV
(columns: Frame,Visibility,X,Y)."""
backend\pipeline\detectors\tracknet_runner.py-239-        points: List[TrajectoryPoint] = []
backend\pipeline\detectors\tracknet_runner.py-240-        with open(csv_win_path, newline="") as
f:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-241-            reader = csv.DictReader(f)
--
backend\eval\fusion_golden.py:34:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py-35-# The 5
person-cue features (fusion_features.PERSON_FEATURE_NAMES).
backend\eval\fusion_golden.py:36:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py-37-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py-38-
backend\eval\fusion_golden.py-39-def _derive_video_path(spec: Dict[str, Any]) -> str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py-40-
"""Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
--
backend\eval\training.py:28:YOLO_FEATURE_NAMES = [
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-29-     "duration",
"peak_velocity", "mean_velocity", "active_frame_count", "track_id_count",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-30-]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-31-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-32-

```

15. user (2026-06-16T03:06:17.787Z)

tests\test_fusion_compare.py

16. assistant (2026-06-16T03:06:23.624Z)

I have everything I need. The existing JSON contract for `\_golden\_features.json` is precisely: `{name, fps, frame\_width, candidates:[{start, end, shuttle{5}, person{5}, audio{3}?, label}], gts:[[a,b]]}`. The proto schema must map onto this exactly, while adding the snapshot-output, versioning, provenance-stripping, and surrogate-id fields the legal memo and harness require.

Let me check the test harness entry point and whether protobuf is already a dep.

17. user (2026-06-16T03:06:25.629Z)

```

---proto files---
---run_all_tests---
run_all_tests.py
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```

python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage

```

Anything after the script name is passed straight through to pytest.

Notes

```

-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport`
package isn't installed (they use `pytest.importorskip`). To exercise them too:
    pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

```

```

from __future__ import annotations

import importlib.util
import subprocess
import sys

def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False

def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]

```

18. assistant (2026-06-16T03:06:28.499Z)

No protobuf dep exists yet and no `.proto` files. This is a greenfield addition. I now have a complete, grounded picture. Let me write up the design and return it via StructuredOutput.

19. user (2026-06-16T03:08:07.277Z)

Structured output provided successfully

20. assistant (2026-06-16T03:08:21.481Z)

I've completed the design and returned it via StructuredOutput. Summary of what I delivered, grounded in the verified infra:

The proto schema (`rally.fixture.v1`) maps 1:1 onto the existing `\*\_golden\_features.json` contract from `backend/eval/fusion\_golden.py` (`name`/`fps`/`frame\_width` + per-candidate `start/end` + `shuttle{5}`/`person{5}`/`audio{3}` feature maps + `label`, plus `gts`), and adds only the fields the harness and legal memo require: `schema\_version`/`label\_version`/`detector\_tag`/`gt\_hash`/`iou\_threshold` (the incommensurability guards), an opaque `fixture\_id` (never the `video\_id` MD5 fingerprint), an `expected` Snapshot, and a `QualityFloor`. Pose/gait/face columns are unrepresentable by design.

Key relevant files:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion\_golden.py` ? the JSON contract the proto mirrors
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion\_compare.py` + `backend\eval\ablation.py` ? the existing video-free CPU re-scoring seam (`load\_videos` ? `VideoCandidates` ? `experiment.compare`) the loader clones
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py` ? `predict`/`compare`/`Variant.spec\_hash`/`\_feature\_column` (the unchanged replay code)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\regression.py` ? `gt\_hash` + the incommensurability guards to reuse
- `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_ablation.py` ? the `skipif` litmus pattern the committed `.pb` fixtures replace (so it runs in CI, not nowhere)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\GOLDEN\_DATA\_SHARING.md` ? the private-OneDrive policy the de-attributed public subset coexists with

Two notes worth flagging: protobuf and `.proto` are greenfield here (no existing dep/files), and `protobuf` is Apache-2.0 so it clears the MIT/Apache/BSD licensing guardrail. The design is purely additive ? it does not modify `experiment.py`/`regression.py`/`ablation.py`/`fusion\_compare.py` internals.